Classifying Binding of Small-Molecule Compounds to SARS-CoV-2 using Machine Learning Models

Team 1
Pradyumna Lanka (Team Lead)
Arianna Daniel (Domain Expert)
Luciano Gnerre
Kiana Beheshtian
Abbas Siddiqui

LLNL-PRES-XXXXXX
This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. Lawrence Livermore National Security, LLC

Lawrence Livermore National Laboratory

Acknowledgments

Hyojin Kim
Brian Gallagher
Suzanne Sindi
Jen Bellig
Cindy Gonzalez
Amar Saini
Mary Silva
Omar DeGuchy
ALL DSC TEAMS

Team 1: Arianna Daniel (Domain Expert)

Degree: PhD in Biology (Immunology, 1st Yr)

Skills: Biology, Beginner in Python

Things learned: LOTS!!! Coding in python, problem solving, leadership, machine learning models.

With more time: I would like to use RDK Fingerprint data and compare differences between molecular descriptor results.

Luciano Gnerre

Degree: B.S in Environmental Systems Science

Skills: Basic Python

Things I learned: The basics of coding in python, ML, and how to improve ML algorithms.

With more time: With more time I would have liked to apply PCA to our work for Task 1, and learn more about Task 2

Kiana Beheshtian

Degree: BA Public Health

Skills: Basic Python

Things learned: Increased familiarity with Python, Concepts of ML, and ML algorithms. Specifically worked on utilizing KNN's for Task 1, worked on creating Random Forest Visualizations for Feature Importance, and became familiar with the concept of Voxelization and Neural Nets.

With more time: I would have liked to increase my knowledge regarding Task 2: writing code for Neural Nets.

Abbas Siddiqui

Degree: Computer Science Engineering, Applied Mathematics with Data Science Emphasis (4th year)

Skills: Machine Learning, Python, Infrastructures for Automating Tasks

Things learned: Neural Nets, Convolutional Neural Nets, Tensorflow, gained familiarity with voxelization, how to methodically approach a complex problem with ML

With more time: Derived a baseline formula for *strong classification* approach, like to explore task two using other models/hybrid models

Team 1: Pradyumna Lanka (Lead)

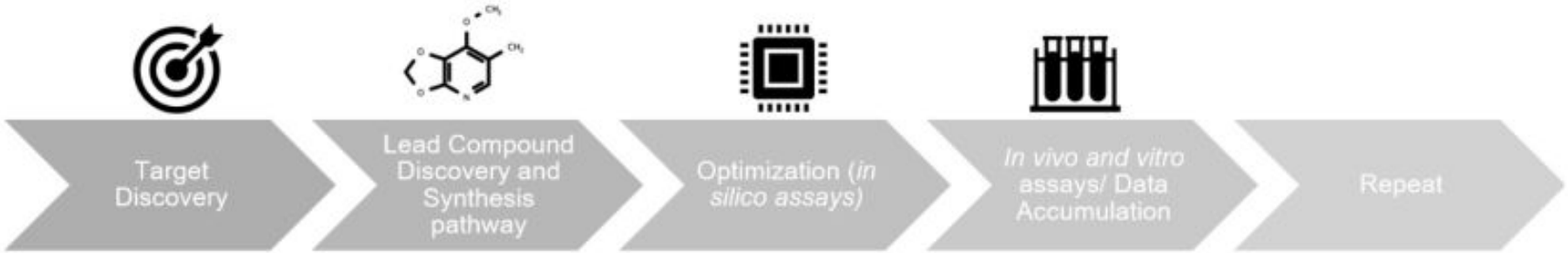
Degree: Ph.D. in Psychological Science (5th year)

Skills: Machine learning, Python

Things learned: Working on a current challenging problem that has real world implications. Managing a team.

With more time: Think about ways to voxelize the data taking into account the atomic radii and the sparsity. Optimize the architecture and parameters of CNNs for task 2. Learn to build fusion models.

Drug Discovery Pipeline



Patel et. al, 2020. *Molecules.* *25*(22), 5277.

1928: Alexander Fleming, accidental Penicillin discovery due to poor sterile technique

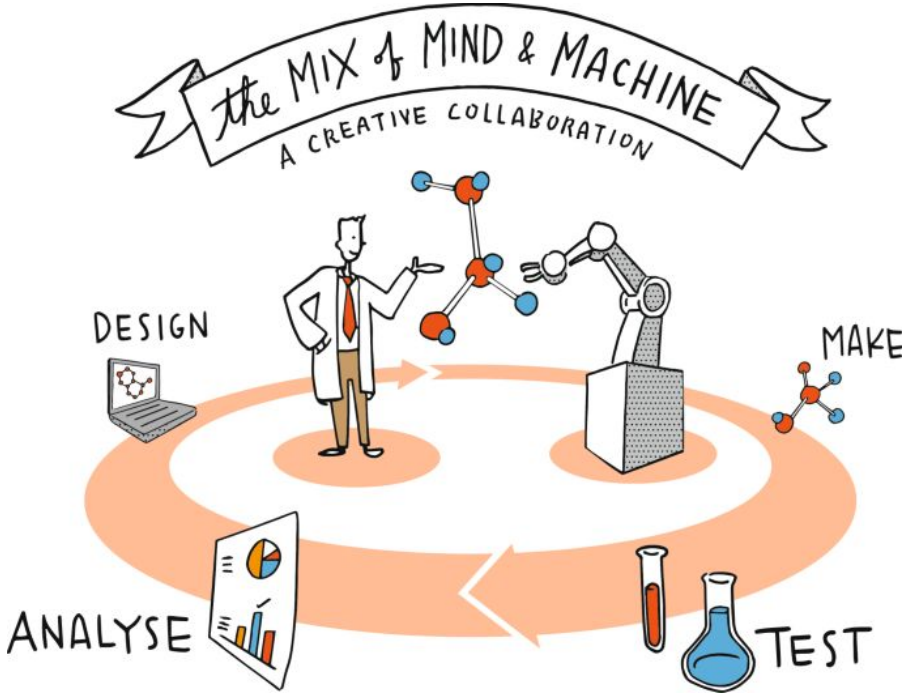
2022: Machine learning and computational methods

"Wet lab" methods are resource-consuming, costly, time consuming, complex and inefficient

"Dry lab" techniques can resolve some of these issues and drawbacks

Machine Learning (ML) for Drug Discovery

- Accelerates the research process
- Allows for virtual screening of LOTS of potential drug compounds
- Faster response for future disease outbreaks

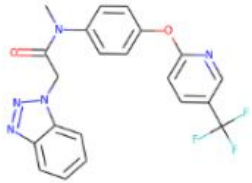


Schneider et al. 2020. *Nat Rev Drug Discov* 19, 353–364.

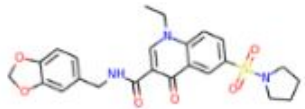
Data Science Challenge Tasks

Task 1:

- Screen ~1955 small-molecule compounds
- 208 molecular features
- Classification: Bind or no-bind



Binding



Non-binding

Task 2:

- 3D atomic ligand information
  - atomic number, XYZ coordinates, charge, valence information etc.
- Use a 3D convolutional neural network to represent ligand data as a 3D voxel grid.

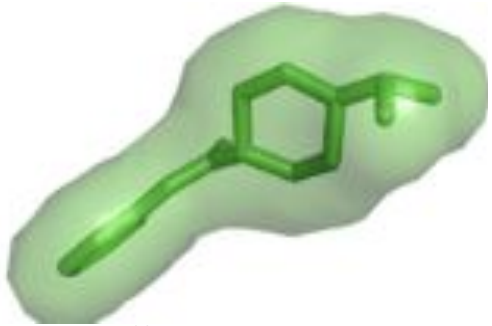
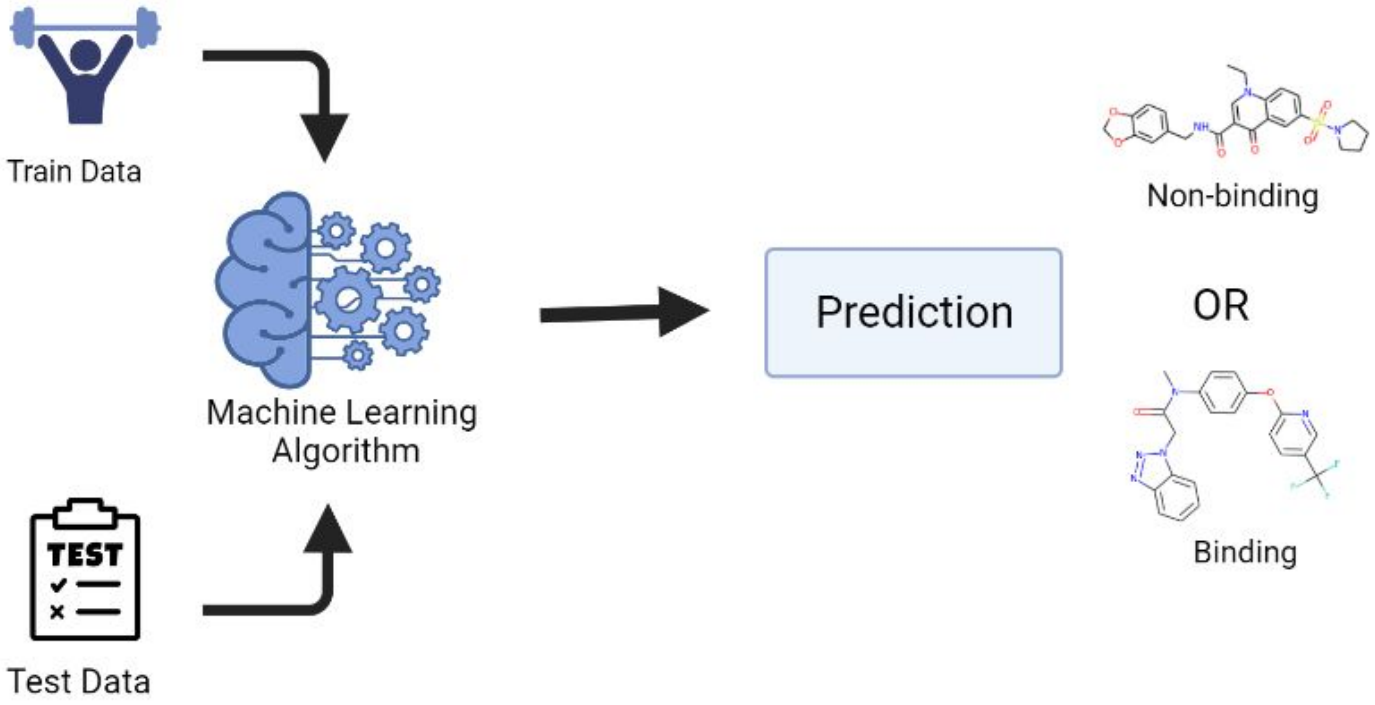


Image Credit https://www.semanticscholar.org

Task 1: Use ML to Classify Small-Molecule Binding to SARS-CoV-2



KNN Algorithm Used in Task 1

What is KNN?

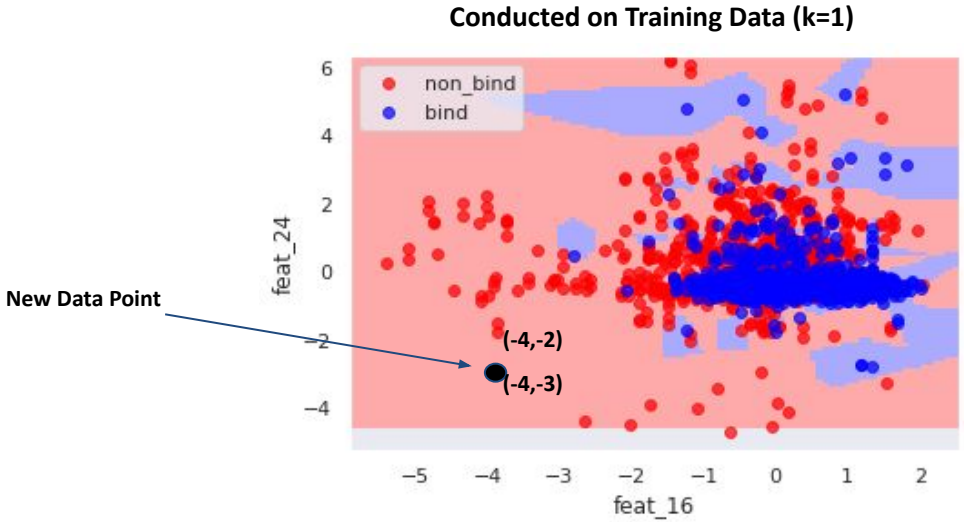
- The k-nearest neighbors (KNN) algorithm, is a ***data classification method*** that helps you predict the category a data point belongs to, based on its proximity to other data points around it.

KNN in 4 Steps

1. Select # of K neighbors
2. Calculate Euclidean Distance to find closest neighbors to new data point

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

3. Count Data Points (neighbors) closest neighbors to new data point
4. Assign New Data Points Category



Performance Metrics for KNN

-How many neighbors should we choose to get best accuracy?….Hyperparameter Tuning?

-The optimal k value was 1

For Test Data:

```
from sklearn.metrics import classification_report #this pertains to TEST data set
#y_test= test.iloc[:,4]
y_pred= knn.predict(x_test)

print(classification_report(y_test, y_pred))
```

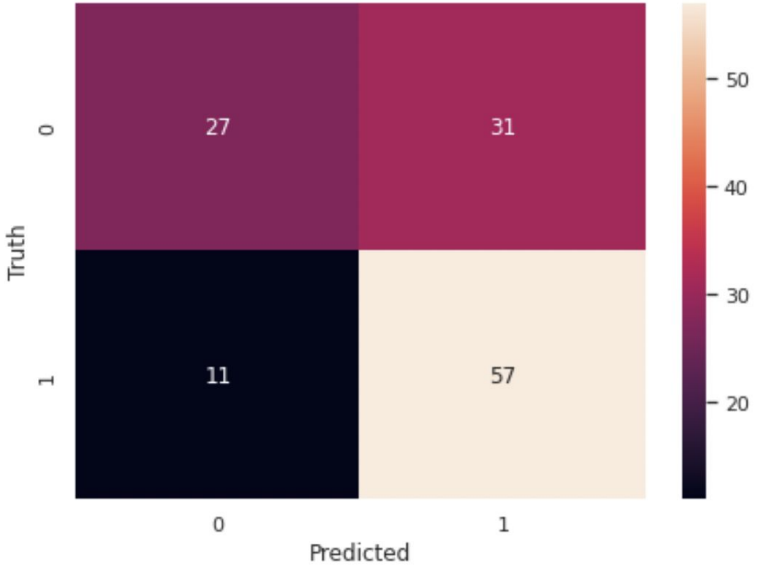
	precision	recall	f1-score	support
0	0.71	0.47	0.56	58
1	0.65	0.84	0.73	68
accuracy			0.67	126
macro avg	0.68	0.65	0.65	126
weighted avg	0.68	0.67	0.65	126

Confusion Matrix Plot for KNN

For Test Data:

TN	FP
FN	TP

0- No Bind
1- Bind



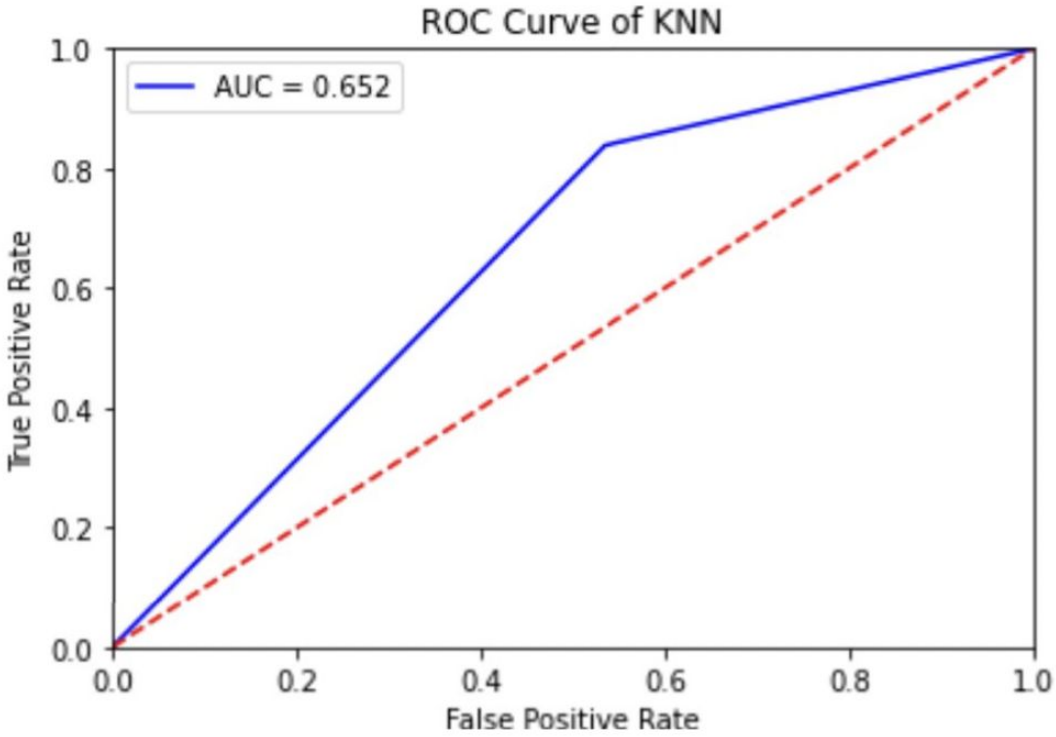
Accuracy: TP+TN/TP+FP+FN+TN= 67%

Precision: TP/TP+FP= 65%

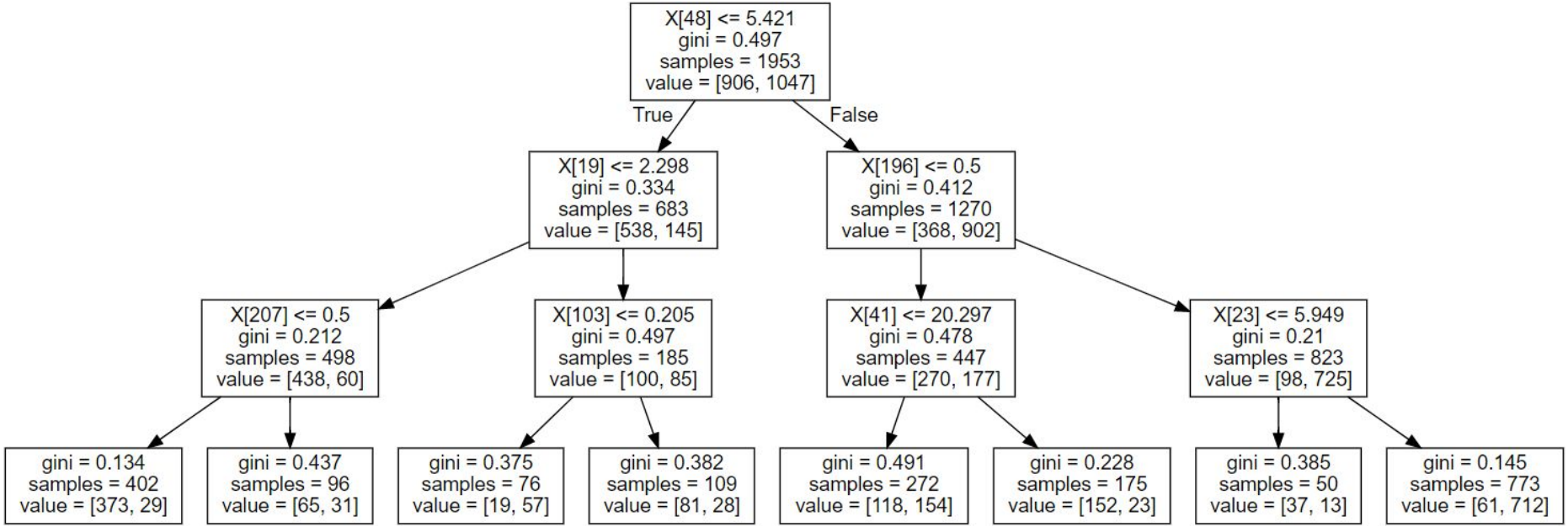
Recall: TP/TP+FN= 84%

F1-Score: 2(Recall*Precision)/(Recall+Precision)= 73 %

ROC Curve Visual for KNN

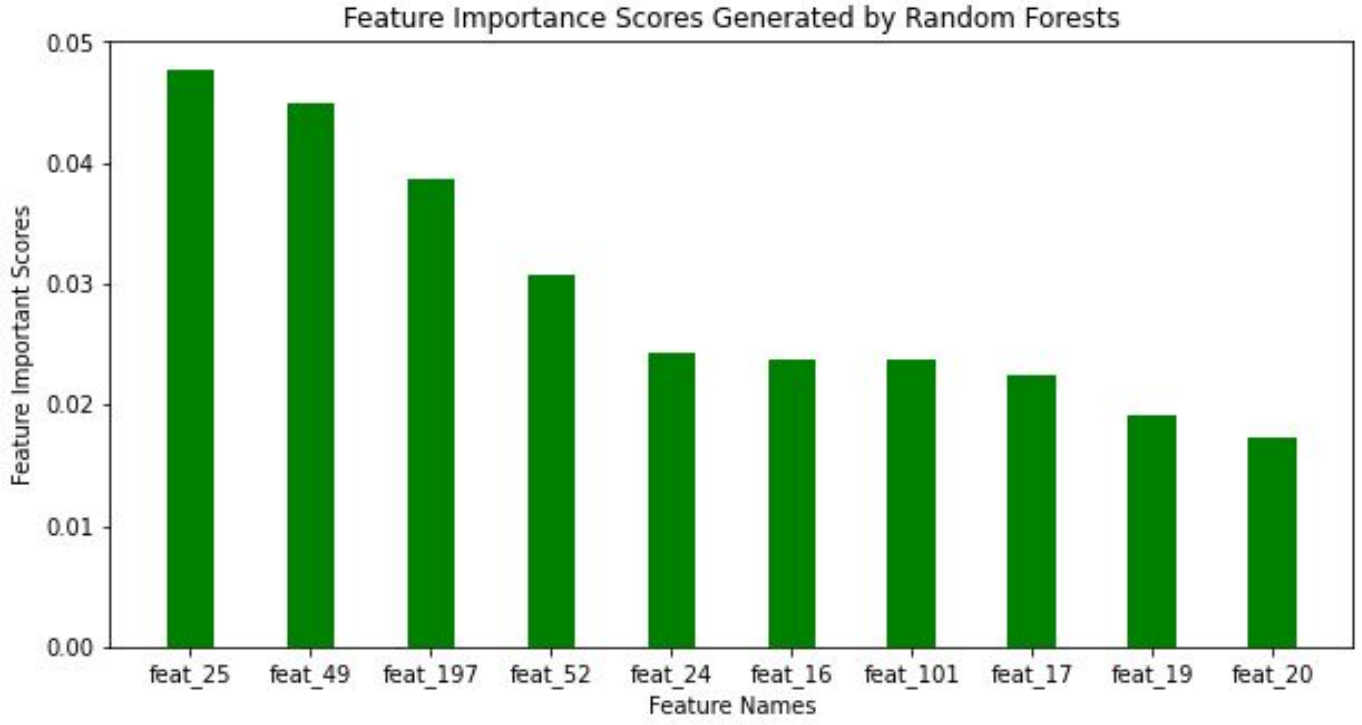


Random Forests



- Results in 76% accuracy when comparing actual vs predicted labels
- Random forest accuracy = 76% vs KNN accuracy= 67%

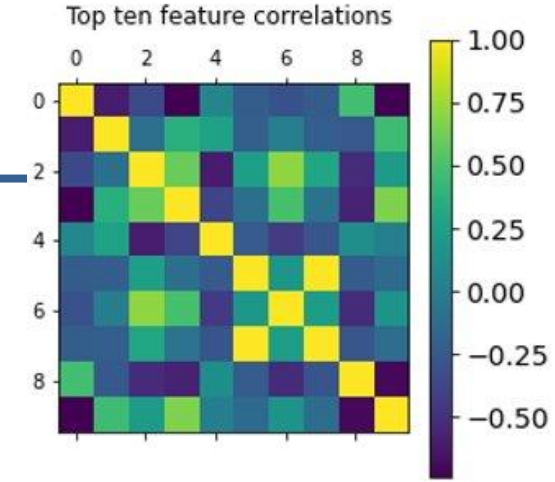
Ranking Feature Importance

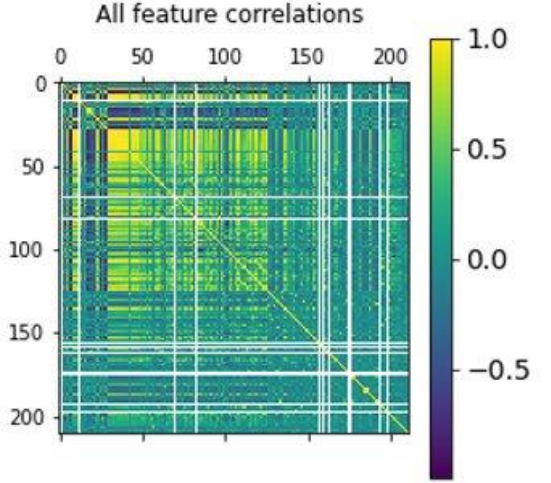


Feature Correlations

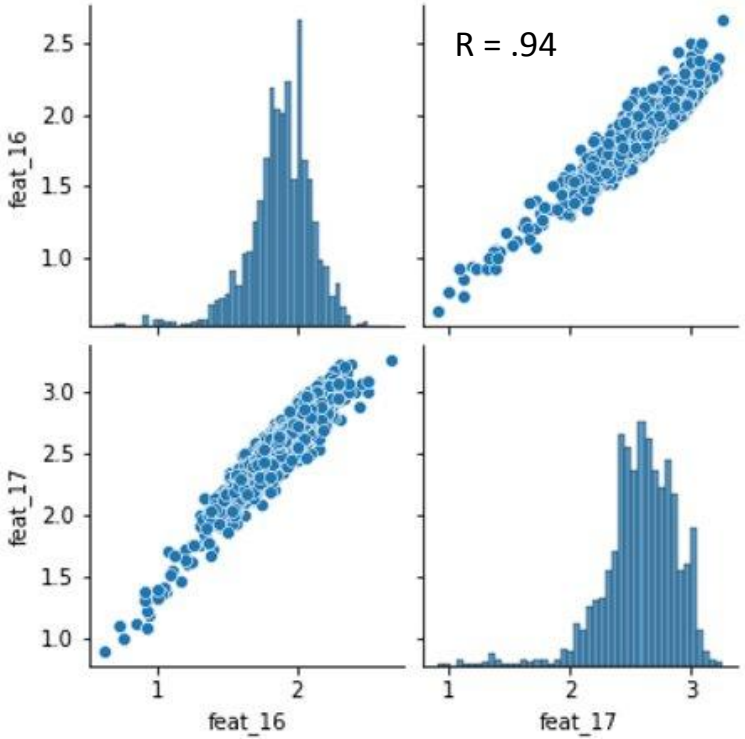
- High correlations imply redundancy between features

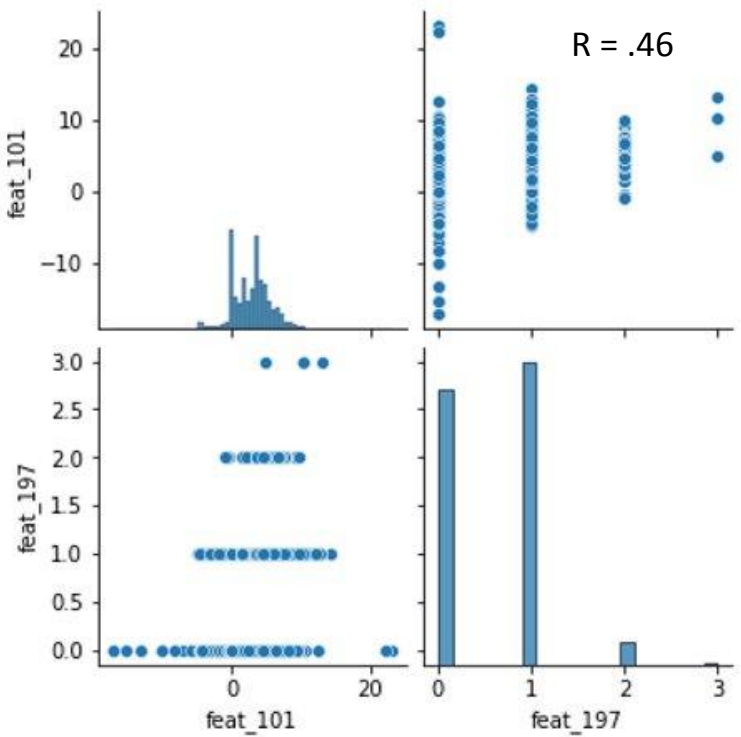
	feat_25	feat_49	feat_197	feat_52	feat_24	feat_16	feat_101	feat_17	feat_19	feat_20
feat_25	1.000000	-0.293931	-0.078187	-0.380951	0.046645	-0.114477	-0.050333	-0.109001	0.198471	-0.381868
feat_49	-0.293931	1.000000	0.022243	0.235731	0.232272	0.002413	0.104950	-0.004696	-0.033017	0.239740
feat_197	-0.078187	0.022243	1.000000	0.419525	-0.161723	0.210416	0.456175	0.271722	-0.153957	0.122686
feat_52	-0.380951	0.235731	0.419525	1.000000	-0.110372	0.019572	0.317030	0.009394	-0.174872	0.390360
feat_24	0.046645	0.232272	-0.161723	-0.110372	1.000000	0.009118	-0.057559	-0.015292	0.072754	0.097553
feat_16	-0.114477	0.002413	0.210416	0.019572	0.009118	1.000000	0.180197	0.944122	-0.050502	-0.016444
feat_101	-0.050333	0.104950	0.456175	0.317030	-0.057559	0.180197	1.000000	0.218384	-0.168874	0.079302
feat_17	-0.109001	-0.004696	0.271722	0.009394	-0.015292	0.944122	0.218384	1.000000	-0.088570	-0.010397
feat_19	0.198471	-0.033017	-0.153957	-0.174872	0.072754	-0.050502	-0.168874	-0.088570	1.000000	-0.359065
feat_20	-0.381868	0.239740	0.122686	0.390360	0.097553	-0.016444	0.079302	-0.010397	-0.359065	1.000000





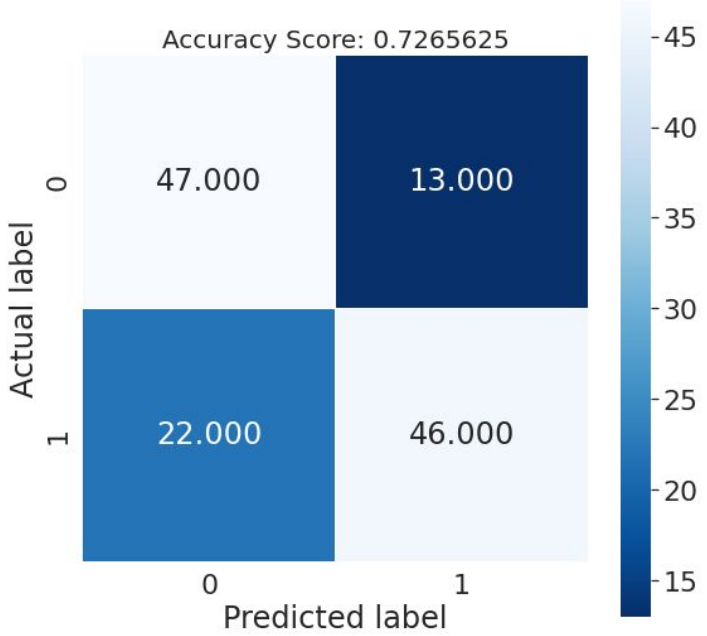
Strongest Correlations Between Top 10 Features





Reducing Dimensionality with PCA

- Principal Component Analysis retains trends in data while reducing dimensionality



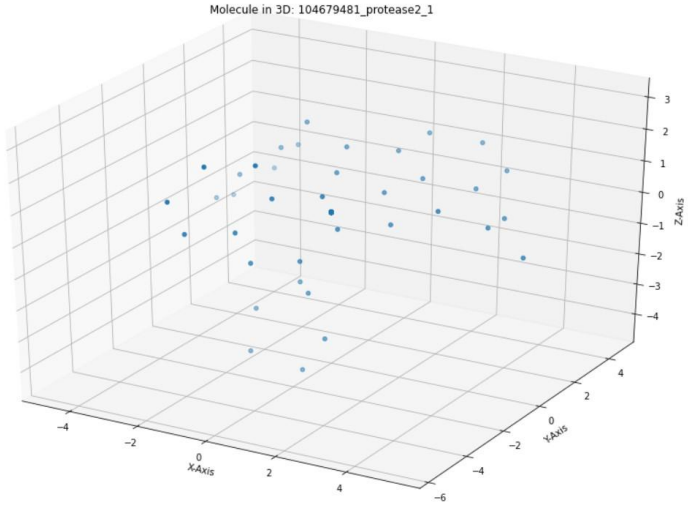
Variance Retained	Number of features	Accuracy
.999	152	.75
.99	105	.7188
.95	75	.6953
.90	60	.6719

Task 2 Stage 1: Data Wrangling

Understand the data and prep it for use in later models

Extract data from HDF files and understand its layout

- Each entry of the HDF files represents a compound
- Multiple poses of the same compounds (Usually 10)
- Each compound links to up to 100 atoms
- Each atom has 22 features
- Features 1 2 3 correspond to x y z
- Features 4 - 13 are one-hot encodings for atom type
- Remaining features are the atoms' attributes



First Compound in Training Set

Note: Most compounds had less than 100 atoms

Voxelization

Voxelization: Process of using spatial data to render a 3D model

19 channels correspond to the 19 other features

Used voxelization to render 3D models of compounds (19, N, N, N)

Reduced runtime using Pickle files

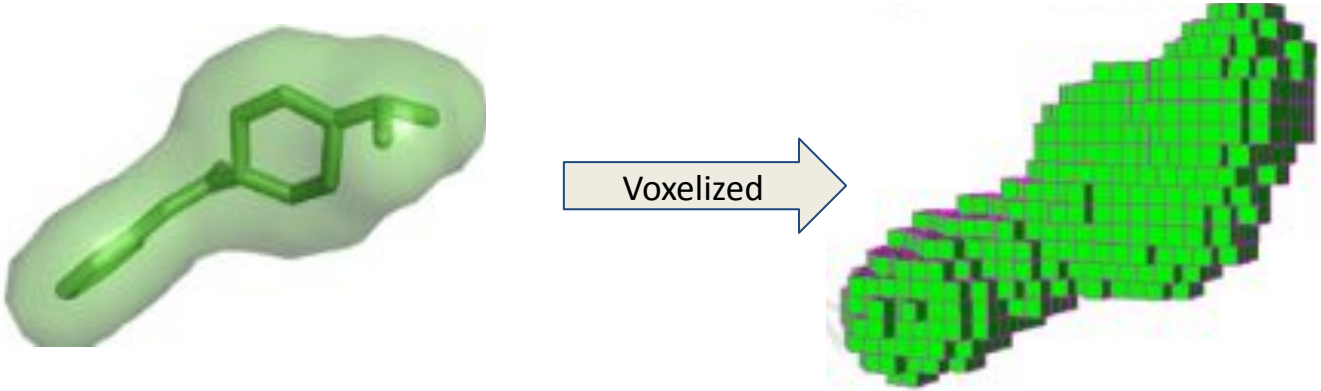


Image Credit https://www.semanticscholar.org

Voxelization: Sparseness vs Granularity

Sparseness: when data structures contain mostly 0s (empty cells)

Granularity: The fineness with which we rendered the voxel

Larger Size:

- Improve granularity while increasing sparseness
- Increases the time needed to train models

Smaller Size:

- Reduce granularity while reducing sparseness
- Reduces the time needed to train models

Experimented with different sizes to find one that was a good balance

Size	% Cells Empty
N = 20	98.75%
N = 16	97.55%
N = 12	94.21%
N = 8	80.47%
N = 5	20%

Optimal: N = 7 70.84%

Note: Above was calculated assuming 100 cells were used

Stage 2: CNN Modeling and Testing

Began to understand the common layers and parameters used in CNNs

Systematically created permutations of a smaller CNN and built intuition by observing their behaviors

Gained better idea of what the CNN is doing

Begin experimenting with CNN architecture and tuning the hyperparameters

Used metrics to follow how well the models are doing

(Tensorboard)

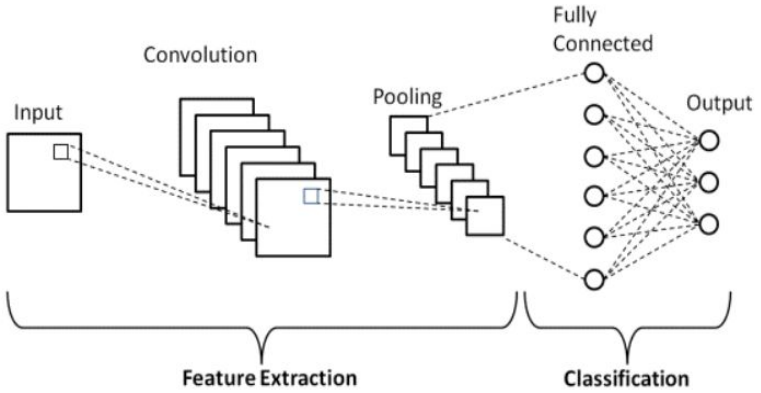


Image Credit: https://www.upgrad.com/blog/basic-cnn-architecture/

Modeling/Testing

```
model = keras.Sequential()
model.add(layers.Conv3D(19, (1, 1, 1), activation='relu', input_shape=(8, 8, 8, 19)))
model.add(layers.AveragePooling3D((2, 2, 2)))

model.add(layers.Flatten())
n = 1
for i in range(n):
    model.add(layers.Dense(5, activation='sigmoid'))
model.add(layers.Dense(2))
```

	Sample	Best
Training	0.738	0.9515
Testing	0.6212	0.7210

```
model.add(layers.Conv3D(19, (2, 2, 2), activation='relu', input_shape=(8, 8, 8, 19)))
model.add(layers.AveragePooling3D((2, 2, 2)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(19, (2, 2, 2), activation='relu'))
model.add(layers.AveragePooling3D((2, 2, 2)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(19, (1, 1, 1), activation='relu'))
model.add(layers.AveragePooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Flatten())

model.add(layers.Dense(5, activation='sigmoid'))
model.add(layers.Dense(2))
```

	Sample	Best
Training	0.5043	0.9790
Testing	0.4566	0.7023

Modeling/Testing

```
model = keras.Sequential()

model.add(layers.Conv3D(19, (2, 2, 2), activation='relu', input_shape=(8, 8, 8, 19)))
model.add(layers.AveragePooling3D((2, 2, 2)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(19*2, (2, 2, 2), activation='relu'))
model.add(layers.AveragePooling3D((2, 2, 2)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(19, (1, 1, 1), activation='relu'))
model.add(layers.AveragePooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Flatten())

model.add(layers.Dense(5, activation='sigmoid'))
model.add(layers.Dense(1, activation ='sigmoid'))
```

	Sample	Best
Training	0.9076	0.9736
Testing	0.6831	0.7023

Modeling/Testing

```
model = keras.Sequential()

model.add(layers.Conv3D(64, (3, 3, 3), activation='relu', input_shape=(n, n, n, 19)))
model.add(layers.MaxPooling3D((2, 2, 2)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(128, (3, 3, 3), kernel_regularizer='L2', activation='relu'))
model.add(layers.MaxPooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(128, (1, 1, 1), activation='relu')) # , kernel_regularizer='L1'
model.add(layers.MaxPooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Flatten())

model.add(layers.Dense(5, activation='relu'))
model.add(layers.Dense(1, activation ='sigmoid'))
```

	Sample	Best
Training	0.9588	0.97559
Testing	0.6804	0.6898

Modeling/Testing

```
model = keras.Sequential()

model.add(layers.Conv3D(64, (1, 1, 1), activation='relu', input_shape=(n, n, n, 19)))
model.add(layers.Conv3D(64, (1, 1, 1), activation='relu'))
model.add(layers.MaxPooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(128, (2, 2, 2), kernel_regularizer='L2', activation='relu'))
model.add(layers.MaxPooling3D((2, 2, 2)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(128, (3, 3, 3), activation='relu')) # , kernel_regularizer='L1'
model.add(layers.MaxPooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Flatten())

model.add(layers.Dense(1, activation ='sigmoid'))
```

	Sample	Best
Training	0.9931	0.9691
Testing	0.7257	0.7265

Modeling/Testing

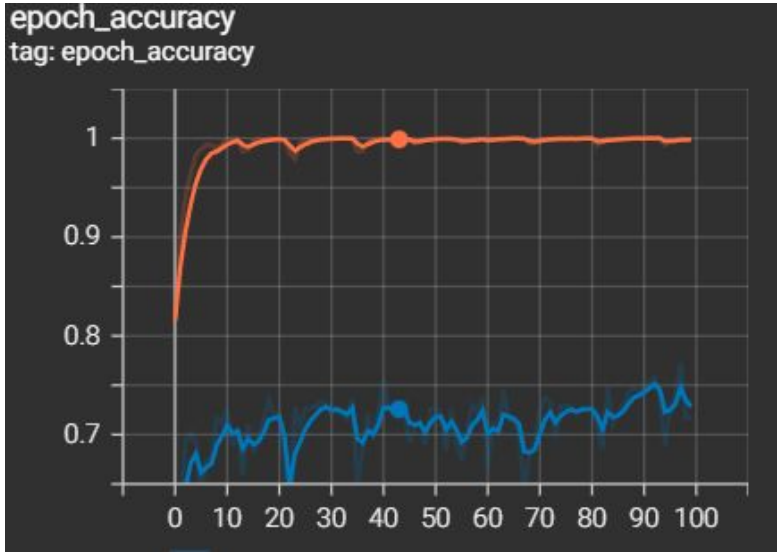
```
N = 7

model = keras.Sequential()

model.add(layers.Conv3D(64, (1, 1, 1), activation='relu', input_shape=(N, N, N, 19)))
model.add(layers.Conv3D(64, (1, 1, 1), activation='relu'))
model.add(layers.MaxPooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(128, (2, 2, 2), kernel_regularizer='L1L2', activation='relu'))
model.add(layers.MaxPooling3D((2, 2, 2)))
model.add(layers.BatchNormalization())

model.add(layers.Conv3D(128, (3, 3, 3), activation='relu')) # , kernel_regularizer='L1'
model.add(layers.MaxPooling3D((1, 1, 1)))
model.add(layers.BatchNormalization())

model.add(layers.Flatten())

model.add(layers.Dense(1, activation ='sigmoid'))
```

	Sample	Best
Training	0.9989	0.9994
Testing	0.7180	0.7555

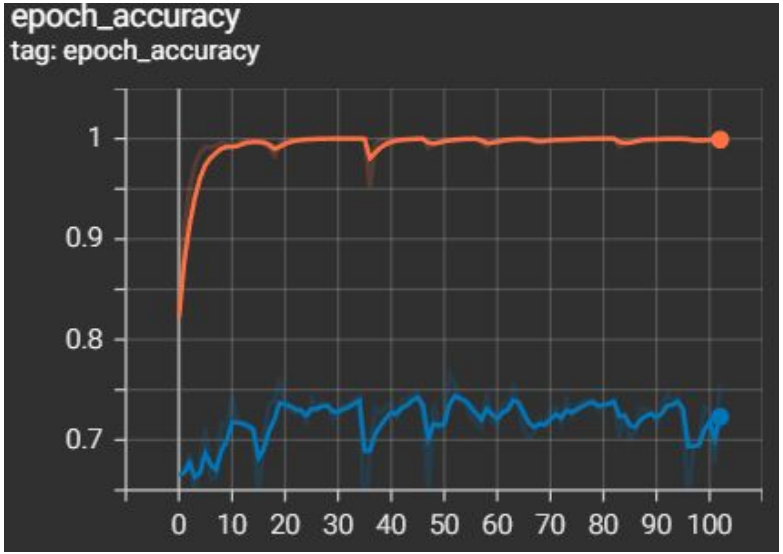
Final Model!

Stage 3: Analysis

Sample Model Accuracy



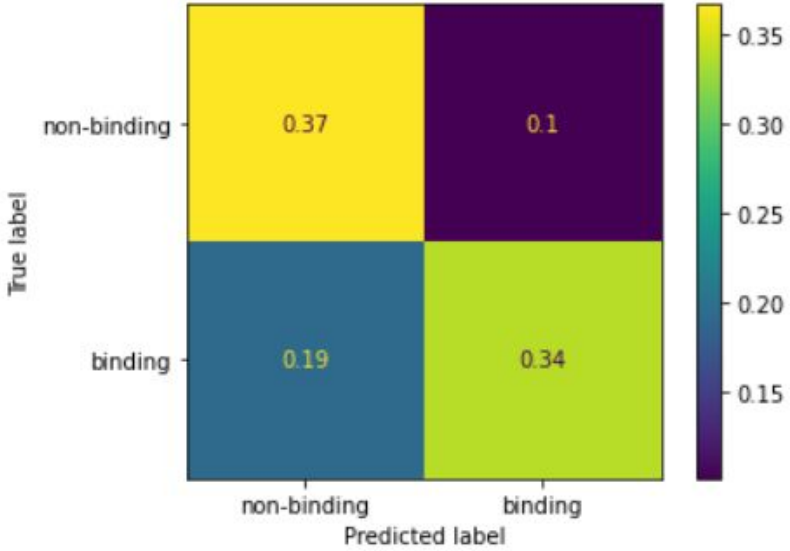
Best Model Accuracy



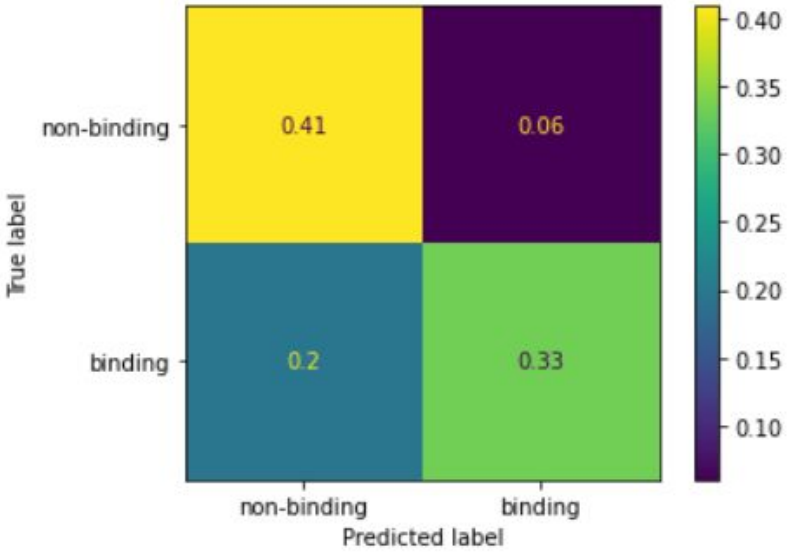
Orange - Training Blue - Test

Analysis

Sample Model Confusion Matrix



Best Model Confusion Matrix



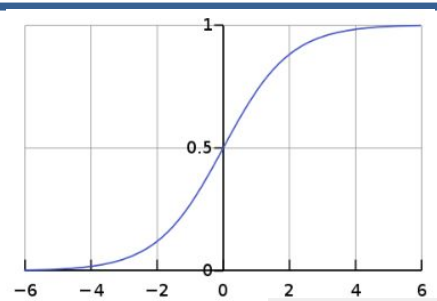
Stage 4: Strong Classification

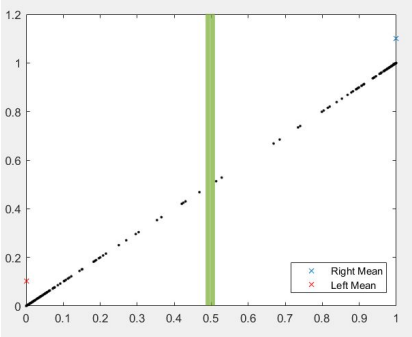
Curtain the classification of points not deemed to be *strongly classified* (Close to 0 or 1)

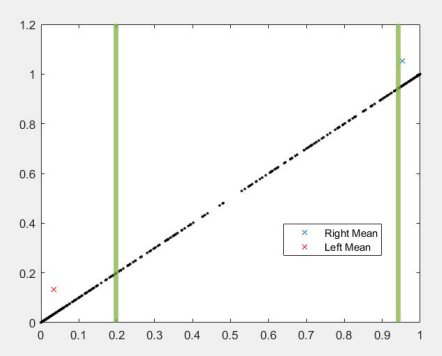
Aim: Reduce the FP rate by filtering points that are within a certain range of 0.5 (very weak)

Procedure: Set bounds s.t. only points below/above them are classified

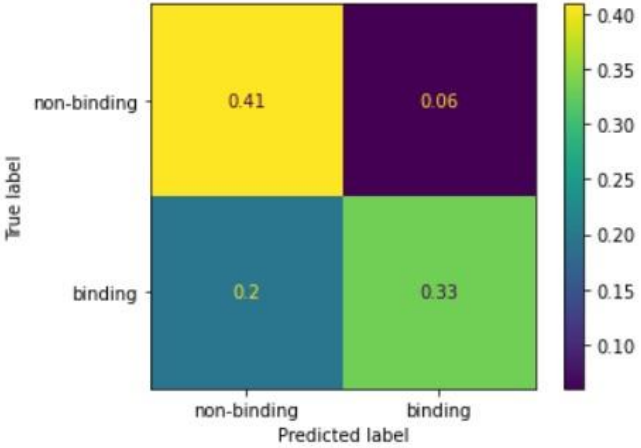
In the figures, when the variance is low, the bounds are more lenient and vice versa when variance is high



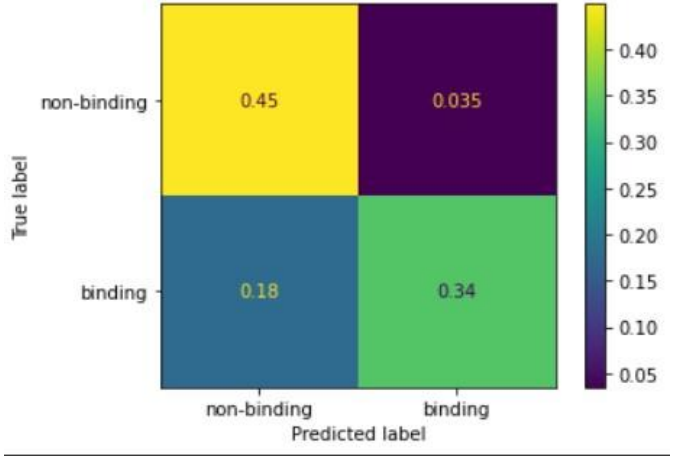




Stage 4: Strong Classification



Of the compounds deemed to bind:
(0.06/(0.33 + 0.06)) * 100 = 15.38%



Of the compounds deemed to bind:
(0.035/(0.34 + 0.035)) * 100 = 9.33%

(1 - (9.33/15.38))*100 = 39.33% Reduction in FP rate

Conclusion

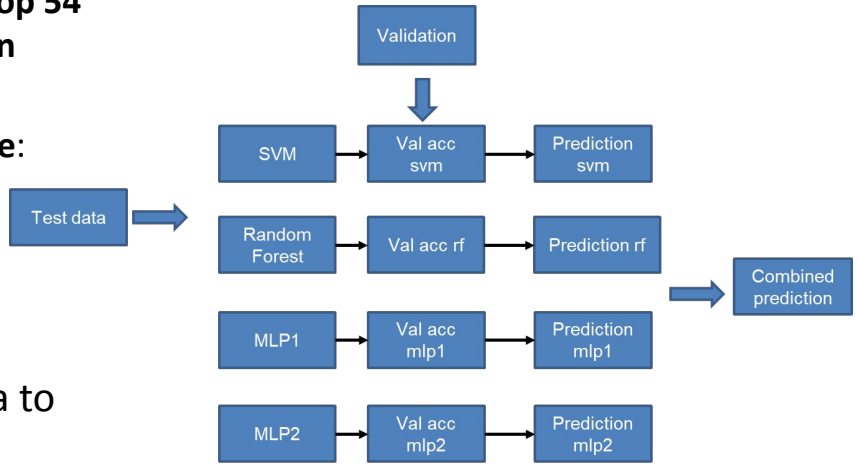
Classifier	Accuracy	Recall	Precision	F1 score	AUC
K-Nearest Neighbors	0.67	0.84	0.65	0.73	0.65
Decision Tree	0.71	0.63	0.78	0.70	0.72
Bagged Trees	0.76	0.62	0.89	0.73	0.77
Random Forest	0.76	0.56	0.97	0.71	0.77
Random Forest (Top 60 feat GINI)	0.76	0.56	0.97	0.71	0.77
SVM RFECV top 54 (grid search)	0.81	0.74	0.89	0.81	0.82
SVM RFECV top 54 (rand search)	0.80	0.74	0.88	0.80	0.81
Random forest RFECV top54	0.77	0.60	0.93	0.73	0.78
MLP (Sklearn) all feat	0.79	0.76	0.83	0.79	0.79
Bagging MLP (Sklearn) all feat	0.77	0.68	0.87	0.76	0.78
MLP (Keras) all feat	0.72	0.69	0.76	0.72	0.72
MLP (Keras) top 54 feat	0.73	0.66	0.80	0.73	0.74
MLP (Keras) All Dropout	0.77	0.71	0.83	0.76	0.77
MLP (Keras) all feat optim	0.75	0.76	0.77	0.76	0.75
Accuracy weighted ensemble	0.73	0.50	1.00	0.67	0.75

RFECV: Recursive feature elimination with cross-validation It gave us 54 features.
SVM: Support vector machines (rbf-kernel)
MLP: Multi-layer perceptron
grid search: searching for optimal hyperparameters along the grid of combinations of specified values
randomized search: Sampling from univariate distribution of hyperparameters to find the optimal hyperparameters.
Bagging: Bootstrap aggregation
Keras: API for TensorFlow to build artificial neural nets in Python.
Dropout: A regularization method that drops units at random to improve generalizability.
Early stopping: Stopping the training early to avoid overfitting the training data. All neural network models implemented here used early stopping.

Conclusion

- **The best performing model for task 1 was**
  - **RBF-kernel Support vector machine (with optimal hyperparameters of C: 10, gamma: 0.01) using the top 54 features obtained from recursive feature elimination with cross-validation**
  - **Accuracy:** 0.81**, Recall:** 0.74, **Precision:** 0.89, **F1 score**: 0.81, **ROC AUC:** 0.82

- We also implemented an **ensemble classifier** which combines the predictive power of all classifiers and weights them by the accuracy on the validation data to make predictions on the test data.
  - **Accuracy:** 0.73, **Recall:** 0.50, **Precision:** 1, **F1 score**: 0.67, **ROC AUC:** 0.75



Ensemble classifier

Lessons learned

- The tasks were hard especially task 2, but the problem was much harder.
- Learned about working with teams:
  - Working with a diverse team with varying levels of experiences.
  - More specifically optimizing both individual members' goals and teams goal.
- Met with a lot of wonderful people both from UC Merced and LLNL and got to hear about their research and life experiences.